INFORME DE LABORATORIO # 3

Gramáticas Libres de Contexto

Diseño y validación de una GLC para un lenguaje de programación simplificado

Asignatura	Teoría de la Computación – INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrantes	José Sáez – Catalina Vergara
Fecha	25 de octubre de 2026

Índice

1. Introducción **2**

2. Desarrollo **2**

2.1. Pregunta 1 – Componentes de la gramática 2

2.2. Pregunta 2 – Construcción de la Gramática Libre de Contexto 4

2.3. Pregunta 3 – Derivaciones y árbol sintáctico 8

2.4. Pregunta 4 – Validación y detección de errores 12

3. Extensión de la gramática **13**

3.1. Nueva sintaxis propuesta 14

3.2. Modificaciones realizadas a la gramática 14

3.3. Ejemplo de cadena válida 14

3.4. Ejemplo de cadena inválida 14

3.5. Derivación de ejemplo 15

4. Conclusiones **15**

Anexo A: evidencia en JFLAP 7.1 **16**

Anexo B: uso de inteligencia artificial **19**

Referencias **19**

1. Introducción

En este laboratorio diseñamos una Gramática Libre de Contexto (GLC) para un lenguaje de programación simplificado: programas con asignaciones, operaciones aritméticas e instrucciones de salida. Identificamos los terminales y no terminales, definimos la gramática formalmente como $G = (V, \Sigma, P, S)$, hicimos derivaciones por la izquierda y árboles sintácticos, y probamos la gramática con el programa de referencia y con ocho cadenas válidas e inválidas. Como extensión agregamos la declaración de variables (`entero x #`, `real temperatura #`). La gramática se cargó en JFLAP 7.1, donde se obtuvieron los árboles sintácticos que se muestran como evidencia (Anexo A).

2. Desarrollo

2.1. Pregunta 1 – Componentes de la gramática

a). Símbolos terminales

Los terminales son los símbolos que aparecen finalmente en las cadenas del lenguaje. A continuación se listan los terminales que hacen falta para representar el lenguaje de referencia, clasificados según la taxonomía solicitada:

Cuadro 1: Símbolos terminales de la gramática.

Categoría	Elementos identificados
Palabras reservadas	`Begin`, `Programa`, `End`, `salida`
Operadores	Aritméticos: `+`, `-`, `*` (multiplicación), `/` (división), `%` (resto o módulo). Asignación: `=`
Símbolos especiales	`#` (terminador de instrucción) `/*` y `*/` (delimitadores del comentario de bloque)
Identificadores	Clase léxica `ID`: *letra* (`a`–`z`, `A`–`Z`) seguida de cero o más *letra*, *dígito* (`0`–`9`) o *guion bajo* (`_`). Ejemplos: `x`, `z2`, `h2`, `temporal`, `operativo_01_02`
Números	Clase léxica `NUM`: uno o más dígitos, opcionalmente seguidos de un punto y más dígitos. Ejemplos: `20`, `10`, `0`, `1.345`

Nota. Los terminales `entero` y `real` no aparecen en la tabla porque los agregamos después, con la extensión creativa (sección 3). Tampoco incluimos los espacios ni los saltos de línea: solo separan palabras y el análisis léxico los descarta.

Nota. En la tabla aparecen `/*` y `*/` porque son los delimitadores de los comentarios, pero en la gramática no son terminales por sí solos. Todo el comentario, con sus delimitadores, se reconoce como un único token `COM`, y ese token es el que está en Σ y en la regla $\langle$ `comentario` $\rangle \to$ `COM`. Así funciona también un analizador léxico real: consume los delimitadores junto con el texto del comentario.

b). Símbolos no terminales

Los no terminales son los símbolos auxiliares que la gramática usa para describir la estructura del lenguaje; todos ellos se reemplazan finalmente por terminales. Los propuestos son:

$\langle$ `programa` $\rangle$, $\langle$ `lista_instr` $\rangle$, $\langle$ `instr` $\rangle$, $\langle$ `asignacion` $\rangle$, $\langle$ `instr_salida` $\rangle$, $\langle$ `declaracion` $\rangle$, $\langle$ `tipo` $\rangle$, $\langle$ `expresion` $\rangle$, $\langle$ `termino` $\rangle$, $\langle$ `op` $\rangle$, $\langle$ `id` $\rangle$, $\langle$ `num` $\rangle$, $\langle$ `comentario` $\rangle$.

c). Símbolo inicial

El símbolo inicial de la gramática es el no terminal que describe un programa completo:

$$\boxed{S = \langle \texttt{programa} \rangle}$$

d). Definición formal de la gramática

La gramática libre de contexto se define formalmente como la cuádrupla $G = (V, \Sigma, P, S)$ [3]:

V **(no terminales)**

=

```
{ programa, lista_instr, instr, asignacion, instr_salida,
declaracion, tipo, expresion, termino, op, id, num, comentario }
```

Σ **(terminales)**

=

```
{ Begin, Programa, End, salida, entero, real, =, #, +, -, *,
/, %, ID, NUM, COM }
```

P **(producciones)**

= el conjunto de 24 reglas que se desarrolla íntegramente en la sección 2.2.

S **(símbolo inicial)**

= `programa`.

En este diseño, `ID`, `NUM` y `COM` son *clases léxicas*: cada una de ellas representa un conjunto infinito de cadenas concretas que el analizador léxico reconoce mediante una expresión regular (sus formas se describen en la tabla de terminales de la parte a) de esta pregunta). La gramática se define, por tanto, sobre el alfabeto de *tokens* y no sobre el alfabeto de caracteres, que es la división habitual entre las fases léxica y sintáctica de un compilador.

e). Análisis: por qué no enumerar identificadores y números

No conviene escribir en la gramática cada identificador y cada número posible, por tres razones:

- **Son infinitos.** Un identificador puede tener cualquier largo (`x`, `xx`, `xxx`, ...) y un número puede tener cualquier cantidad de cifras. Para enumerarlos haría falta una cantidad infinita de producciones, y una gramática tiene que ser finita.
- **Al análisis sintáctico no le importa cuál es.** Lo único que necesita saber es que en ese lugar va un identificador o un número: `x` y `temporal` se usan exactamente igual dentro de una instrucción.
- **Ya hay una herramienta mejor.** La forma de un identificador o de un número se puede describir con una expresión regular [1], y eso lo resuelve el análisis léxico.

Por eso modelamos `ID` y `NUM` como tokens y usamos solo dos reglas, $\langle \texttt{id} \rangle \to$ `ID` y $\langle \texttt{num} \rangle \to$ `NUM`. La forma concreta de cada uno se resuelve en la fase léxica.

2.2. Pregunta 2 – Construcción de la Gramática Libre de Contexto

a). Producciones propuestas

Las reglas de producción que definen la GLC son las siguientes. Este tipo de reglas es el que se usa para especificar la sintaxis de los lenguajes de programación, por ejemplo con la notación BNF [4]. El símbolo inicial es ⟨programa⟩ y la gramática queda completa con 24 reglas (las reglas 6, 10, 11 y 12 corresponden a la extensión creativa, incorporada en la sección 3).

Cuadro 2: Conjunto de producciones P.

Nº	**Regla**	**Aplicación**
1	⟨ programa ⟩ → Begin Programa ⟨ id ⟩ ⟨ lista_instr ⟩ End	Estructura del programa
2	⟨ lista_instr ⟩ → ⟨ instr ⟩ ⟨ lista_instr ⟩	Lista recursiva
3	⟨ lista_instr ⟩ → ⟨ instr ⟩	Caso base de la lista
4	⟨ instr ⟩ → ⟨ asignacion ⟩	Asignación
5	⟨ instr ⟩ → ⟨ instr_salida ⟩	Salida
6	⟨ instr ⟩ → ⟨ declaracion ⟩	Declaración (extensión)
7	⟨ instr ⟩ → ⟨ comentario ⟩	Comentario de bloque
8	⟨ asignacion ⟩ → ⟨ id ⟩ = ⟨ expresion ⟩ #	Asignación simple o con expresión
9	⟨ instr_salida ⟩ → salida ⟨ id ⟩ #	Salida de un valor
10	⟨ declaracion ⟩ → ⟨ tipo ⟩ ⟨ id ⟩ #	Declaración de variable (extensión)
11	⟨ tipo ⟩ → entero	Tipo entero (extensión)
12	⟨ tipo ⟩ → real	Tipo real (extensión)
13	⟨ expresion ⟩ → ⟨ termino ⟩	Un solo operando
14	⟨ expresion ⟩ → ⟨ termino ⟩ ⟨ op ⟩ ⟨ expresion ⟩	Con operación (recursión derecha)
15	⟨ termino ⟩ → ⟨ id ⟩	Operando identificador
16	⟨ termino ⟩ → ⟨ num ⟩	Operando numérico
17	⟨ op ⟩ → +	Suma
18	⟨ op ⟩ → -	Resta
19	⟨ op ⟩ → *	Multiplicación
20	⟨ op ⟩ → /	División
21	⟨ op ⟩ → %	Módulo
22	⟨ id ⟩ → ID	Clase léxica identificador
23	⟨ num ⟩ → NUM	Clase léxica número
24	⟨ comentario ⟩ → COM	Clase léxica comentario

b). Explicación de los no terminales

Cuadro 3: Función de cada no terminal dentro de la gramática.

No terminal	**Función dentro de la gramática**
programa	Símbolo inicial. Describe el programa completo: cabecera `Begin Programa` ⟨ `id` ⟩, lista de instrucciones y cierre `End`.
lista_instr	Lista de una o más instrucciones. Usa recursión derecha, así no necesita la producción vacía ε.
instr	Una instrucción del programa: asignación, salida, declaración (extensión) o comentario.
asignacion	Instrucción ⟨ `id` ⟩ `=` ⟨ `expresion` ⟩ `#`. Cubre la asignación simple (`x = 20 #`) y la que tiene operación (`z = x + y #`).
instr_salida	Instrucción de salida: `salida` seguida de un identificador y `#` (p. ej. `salida x #`).
declaracion	Declaración de variable (extensión): un ⟨ `tipo` ⟩, un ⟨ `id` ⟩ y `#`.
tipo	Los tipos declarables: `entero` y `real`.
expresion	Expresión aritmética: un operando, o un operando seguido de un operador y otra expresión (recursión derecha, no ambigua).
termino	Operando de una expresión: un identificador o un número. La gramática no modela precedencia de operadores.
op	Los cinco operadores permitidos: `+`, `-`, `*`, `/` y `%`.
id	Identificador (clase léxica `ID`). Marca dónde termina el análisis léxico y empieza el sintáctico.
num	Número (clase léxica `NUM`).
comentario	Comentario de bloque (clase léxica `COM`), como el del programa de referencia.

Observación sobre los comentarios. Como ⟨comentario⟩ es una alternativa de ⟨instr⟩, la gramática acepta un programa formado solo por comentarios. El enunciado no lo excluye y, en un compilador real, el comentario se descarta en el análisis léxico antes de llegar al análisis sintáctico, por lo que esta situación no se presentaría.

c). Diferencia entre los tres tipos de instrucción

La siguiente tabla distingue explícitamente la asignación simple, la asignación con expresión aritmética y la instrucción de salida:

Tipo	**Reglas usadas**	**Ejemplo**	**Rasgo distintivo**
Asignación simple	⟨asignacion⟩ → ⟨id⟩ = ⟨expresion⟩ #; ⟨expresion⟩ → ⟨termino⟩	x = 20 #	La expresión es un solo operando, sin operador.
Asignación con expresión aritmética	⟨asignacion⟩ → ⟨id⟩ = ⟨expresion⟩ #; ⟨expresion⟩ → ⟨termino⟩ ⟨op⟩ ⟨expresion⟩	z = x + y #	La expresión tiene al menos un operador de {+ - * / %}.
Salida	⟨instr_salida⟩ → salida ⟨id⟩ #	salida z #	Empieza con salida; no lleva = ni expresión.

Cuadro 4: Diferencia entre asignación simple, asignación con expresión y salida.

d). Verificación de la gramática con el programa de referencia

Comprobamos que la gramática propuesta **permite representar el programa de referencia completo**, descomponiéndolo línea a línea según la regla 1 (⟨programa⟩ → Begin Programa ⟨id⟩ ⟨lista_instr⟩ End):

```
Begin
Programa operativo_01_02 -> <programa> -> Begin Programa <id> ...

x = 1.345 # -> <instr> -> <asignacion>
y = 20 # -> <id> '=' <expresion> '#'
                                    -> <num> -> NUM
/* operaciones y salidas */ -> <instr> -> <comentario> -> COM

z=x+y# -> <instr> -> <asignacion>
z2 = x + y # -> <expresion> -> <termino> <op> <expresion>
h=z-x# -> <op> -> '+' | '-' | '*' | '/' | '%'
h2 = z - x #
k=h*y# -> <termino> -> <id> | <num>
t=k/z#
m = k % 10 #
salida z # -> <instr> -> <instr_salida> -> 'salida' <id> '#'
salida h #
salida k #
salida t #
salida m #
temporal = 0 # -> <instr> -> <asignacion>
temporal = x #
x=y#
y = temporal #
salida x #
salida y #
End
```

Listing 1: Programa de referencia y su descomposición en la gramática.

Las 21 instrucciones del programa de referencia encajan en la regla ⟨ lista_instr ⟩ → ⟨ instr ⟩ ⟨ lista_instr ⟩, usando el caso base ⟨ instr ⟩ para la última, y cada una se genera con la regla que le corresponde. La descomposición del Listing 1 muestra que la gramática permite representar el programa completo.

También creemos que la gramática es **no ambigua**, aunque no lo demostramos formalmente. La idea es que en cada punto basta mirar el siguiente token para saber qué regla aplicar. Por ejemplo, ⟨ instr ⟩ se decide según empiece con `salida`, `entero`, `real`, un comentario o un identificador, y ⟨ expresion ⟩ sigue si viene un operador y termina si viene `#`. Con eso cada cadena tiene una única derivación por la izquierda, y por eso el árbol de la sección 2.3 c) es único.

e). Traducción de la GLC al formato de JFLAP 7.1

JFLAP 7.1 [2] solo admite símbolos de *un único carácter* y trata como variable (no terminal) a toda letra mayúscula. Por eso no se pueden escribir las palabras reservadas del lenguaje (`Begin`, `Programa`, `End`) tal como son: si ponemos `S` → `Begin Programa`, JFLAP lo lee como `S` → `B e g i n P r o g r a m a`, con `B` y `P` como no terminales y el resto de las letras como terminales.

Para poder usar la herramienta, codificamos el alfabeto con un carácter por token, sin cambiar la estructura de la gramática del informe. La correspondencia es:

Cuadro 5: Correspondencia entre la notación del informe y la de JFLAP 7.1.

Token del informe	JFLAP	Token del informe	JFLAP
Begin	b	=	=
Programa	p	#	#
End	e	+	+
salida	o	-	-
entero	t	*	*
real	r	/	/
ID	w	%	%
NUM	n		
COM	x		

Cuadro 6: Correspondencia entre los no terminales del informe y los de JFLAP 7.1.

Informe	JFLAP	Informe	JFLAP	Informe	JFLAP
⟨ programa ⟩	S	⟨ termino ⟩	M	⟨ op ⟩	Z
⟨ lista_instr ⟩	L	⟨ id ⟩	J	⟨ num ⟩	Q
⟨ instr ⟩	I	⟨ expresion ⟩	G	⟨ comentario ⟩	C
⟨ asignacion ⟩	A	⟨ instr_salida ⟩	V		
⟨ declaracion ⟩	D	⟨ tipo ⟩	K		

Con esta codificación, las 24 reglas del informe se escriben en JFLAP como:

```
S -> bpJLe (programa) L -> IL | I (lista_instr)
I -> A | V | D | C (instr)
A -> J=G# (asignacion) V -> oJ# (instr_salida)
D -> KJ# (declaracion) K -> t | r (tipo)
G -> M | MZG (expresion) M -> J | Q (termino)
Z -> + | - | * | / | % (op)
J -> w (id) Q -> n (num)
C -> x (comentario)
```

Listing 2: La GLC del informe en el formato de JFLAP 7.1 (archivo `GLC_Lab3.jff`).

La estructura de la gramática es idéntica a la del informe: sólo cambian los nombres de los símbolos. Gracias a esta equivalencia, las cadenas del informe se traducen a cadenas de tokens: la instrucción `z = x + y #` se traduce a `w=w+w#`, y el programa `Begin Programa demo z = x + y # End` se traduce a `bpww=w+w#e`. Como el archivo entregado parte del símbolo inicial `S` (⟨ `programa` ⟩), la cadena que JFLAP 7.1 puede analizar directamente con el archivo `GLC_Lab3.jff` es la del programa completo; para analizar una instrucción suelta hay que cambiar antes el símbolo inicial de la gramática (véase la sección 2.3 c)).

2.3. Pregunta 3 – Derivaciones y árbol sintáctico

Una *derivación por la izquierda* es una secuencia de pasos en la que, en cada uno, reemplazamos el *no terminal más a la izquierda* de la forma sentencial por el lado derecho de alguna de sus reglas. Seguimos así hasta que quedan solo terminales. Si la cadena que obtenemos es la que buscábamos, la derivación demuestra que esa cadena **pertenece** al lenguaje generado por la gramática.

Convención. Como el símbolo inicial es ⟨ `programa` ⟩, en sentido estricto el lenguaje $L(G)$ está formado solo por programas completos. Las cadenas sueltas, como `x = 10 #`, las evaluamos como *instrucciones*, es decir, las derivamos desde ⟨ `instr` ⟩, y los programas completos los derivamos desde ⟨ `programa` ⟩. Por eso, cuando respondemos «Sí», queremos decir que la cadena es una instrucción válida de la gramática.

a). Derivación por la izquierda para `z = x + y #`

La cadena pedida es una instrucción, por lo que se deriva a partir del no terminal ⟨ `instr` ⟩, que es justamente el que describe una instrucción del programa:

$$
\begin{aligned}
\langle\,\texttt{instr}\,\rangle &\Rightarrow \langle\,\texttt{asignacion}\,\rangle \\
&\Rightarrow \langle\,\texttt{id}\,\rangle\ \texttt{=}\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \langle\,\texttt{termino}\,\rangle\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \langle\,\texttt{id}\,\rangle\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \texttt{x}\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \texttt{x}\ \texttt{+}\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \texttt{x}\ \texttt{+}\ \langle\,\texttt{termino}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \texttt{x}\ \texttt{+}\ \langle\,\texttt{id}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{z}\ \texttt{=}\ \texttt{x}\ \texttt{+}\ \texttt{y}\ \texttt{\#}
\end{aligned}
$$

En cada paso se reemplaza efectivamente el no terminal más a la izquierda de la forma sentencial: ⟨ `instr` ⟩, luego ⟨ `asignacion` ⟩, luego ⟨ `id` ⟩, luego ⟨ `expresion` ⟩, luego ⟨ `termino` ⟩, luego ⟨ `id` ⟩, luego ⟨ `op` ⟩, y así sucesivamente. Obsérvese que en los pasos 3, 6 y 10 el no terminal ⟨ `id` ⟩ se instancia con los lexemas concretos `z`, `x` e `y`: esa es la información que aporta el analizador léxico, y a partir de ese momento la derivación es puramente sintáctica.

La misma cadena, en el contexto completo de un programa, se obtiene derivando desde el símbolo inicial:

$$
\begin{aligned}
\langle\,\texttt{programa}\,\rangle &\Rightarrow \texttt{Begin Programa}\ \langle\,\texttt{id}\,\rangle\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{instr}\,\rangle\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{asignacion}\,\rangle\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{id}\,\rangle\ =\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \langle\,\texttt{termino}\,\rangle\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \langle\,\texttt{id}\,\rangle\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \texttt{x}\ \langle\,\texttt{op}\,\rangle\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \texttt{x}\ +\ \langle\,\texttt{expresion}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \texttt{x}\ +\ \langle\,\texttt{termino}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \texttt{x}\ +\ \langle\,\texttt{id}\,\rangle\ \texttt{\#}\ \texttt{End}\\
&\Rightarrow \texttt{Begin Programa demo z}\ =\ \texttt{x}\ +\ \texttt{y}\ \texttt{\#}\ \texttt{End}
\end{aligned}
$$

b). Derivación por la izquierda para `salida z #`

$$
\begin{aligned}
\langle\,\texttt{instr}\,\rangle &\Rightarrow \langle\,\texttt{instr_salida}\,\rangle\\
&\Rightarrow \texttt{salida}\ \langle\,\texttt{id}\,\rangle\ \texttt{\#}\\
&\Rightarrow \texttt{salida z \#}
\end{aligned}
$$

El árbol correspondiente se muestra en la Figura 1. La misma cadena de reglas (⟨instr⟩ → ⟨instr_salida⟩ → `salida` ⟨id⟩ `#`) produce un árbol más pequeño, porque la instrucción de salida no contiene expresión aritmética.

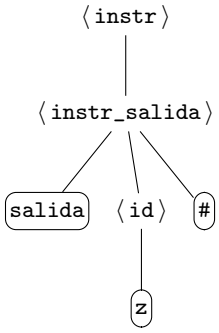


Figura 1: Árbol de derivación de la cadena `salida z #`.

c). Árbol de derivación

El árbol de derivación de `z = x + y #` está en la Figura 2. Lo construimos a partir de la derivación por la izquierda que hicimos antes, así que es consistente con la gramática de la sección 2.2.

En JFLAP 7.1 se puede obtener el mismo árbol, pero la herramienta tiene un detalle que hay que tener en cuenta. El símbolo inicial de la gramática cargada es `programa` (en la codificación de la Tabla 6, el `S`), por lo que una cadena de *instrucción* suelta no se puede analizar partiendo desde `S`. Comprobamos que JFLAP 7.1 toma como símbolo inicial el *lado izquierdo de la primera producción* del archivo `.jff`, ya que el formato no permite declararlo de otra forma. Por eso basta

con reordenar las reglas para analizar una instrucción aislada. Hay dos formas de obtener el árbol en la herramienta:

- **Árbol de la instrucción.** Abrimos el archivo auxiliar `GLC_Lab3_instr.jff`, que tiene las mismas 24 reglas pero con `I` → `A` como primera producción. Así, el símbolo inicial de la herramienta pasa a ser el no terminal de las instrucciones (`I` en la codificación de la Tabla 6). Luego analizamos la cadena de tokens `w=w+w#`, que equivale a `z = x + y #`. JFLAP 7.1 responde *"String accepted!"* y el árbol resultante es el de la Figura 2.
- **Árbol del programa.** Mantenemos `S` como símbolo inicial, abriendo `GLC_Lab3.jff`, y analizamos la cadena de tokens del programa: `bpww=w+w#e`, que equivale a `Begin Programa demo z = x + y # End`. El árbol resultante es el de la Figura 3. Hay que advertir que el algoritmo de *brute force* de JFLAP 7.1 tiene costo exponencial respecto del largo de la cadena. Con este programa de diez tokens, la búsqueda superó los 50 000 nodos sin terminar después de varios minutos. Por eso la evidencia de la Figura 6 usa el programa mínimo equivalente `bpww=w#e` (`Begin Programa demo w = w # End`), que JFLAP resuelve en menos de dos segundos.

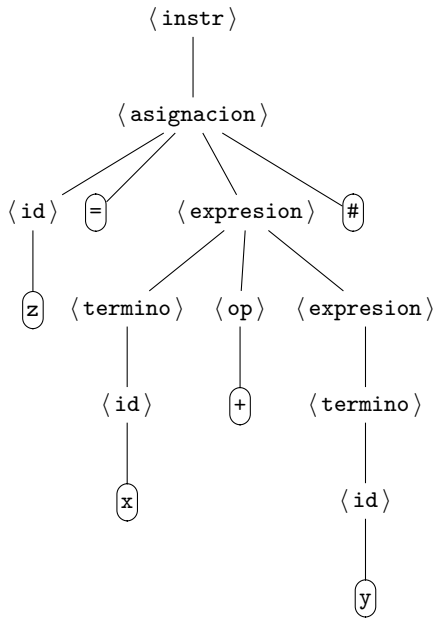


Figura 2: Árbol de derivación de la cadena `z = x + y #`. Los nodos entre ángulos son no terminales; las hojas dentro de recuadros son los terminales (lexemas concretos `z`, `x` y `y`).

Para mostrar el árbol completo, con la cabecera del programa, la Figura 3 presenta la derivación desde el símbolo inicial `Begin Programa demo z = x + y # End`.

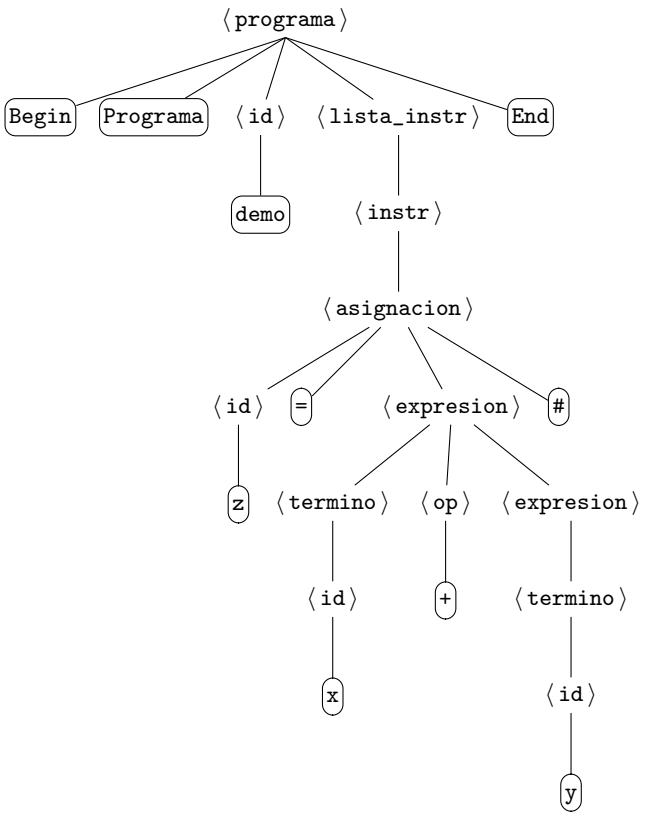


Figura 3: Árbol de derivación del programa `Begin Programa demo z = x + y # End`.

d). Explicación del árbol

La relación entre la raíz, los no terminales, los terminales y las hojas es la siguiente:

- **Raíz.** El nodo raíz es ⟨ `instr` ⟩ (o ⟨ `programa` ⟩ en el árbol completo). Representa el no terminal del que parte la derivación y, por lo tanto, identifica *qué construcción sintáctica* se está reconociendo: una instrucción del programa.
- **No terminales (nodos internos).** Cada nodo interno es un no terminal de la gramática y cada uno se desarrolla con una regla distinta: ⟨ `instr` ⟩ → ⟨ `asignacion` ⟩; ⟨ `asignacion` ⟩ → ⟨ `id` ⟩ `=` ⟨ `expresion` ⟩ `#`; ⟨ `expresion` ⟩ → ⟨ `termino` ⟩ ⟨ `op` ⟩ ⟨ `expresion` ⟩; ⟨ `termino` ⟩ → ⟨ `id` ⟩; y así sucesivamente. La estructura del árbol es *exactamente* la estructura anidada de las reglas, por lo que el árbol es una representación visual de la derivación.
- **Terminales (hojas).** Las hojas son los símbolos terminales que ya no se pueden derivar: `z`, `x`, `y`, `=`, `+` y `#`. Los tres primeros son instancias de la clase léxica `ID` y los tres últimos son terminales fijos. Cada hoja ocupa una posición fija en la cadena final: la concatenación de las hojas, leyéndolas de izquierda a derecha, da exactamente la cadena analizada: `z = x + y #`. Por eso el árbol es la prueba de que esa cadena pertenece al lenguaje.

La raíz indica el contexto sintáctico, *los nodos internos son los no terminales y su disposición muestra qué regla se aplicó en cada paso. Las hojas son los terminales*, y *su concatenación es la cadena.* Como la gramática es no ambigua (sección 2.2), este árbol es único.

e). Validación de otras cadenas

Cuadro 7: Cadenas adicionales y su posibilidad de ser generadas por la gramática.

Cadena	¿Puede generarse?	Justificación
`x = 20 #`	**Sí**	Asignación simple: ⟨ `instr` ⟩ → ⟨ `asignacion` ⟩ → ⟨ `id` ⟩ `=` ⟨ `expresion` ⟩ `#`, con `x` como ⟨ `id` ⟩ y ⟨ `expresion` ⟩ → ⟨ `termino` ⟩ → ⟨ `num` ⟩ instanciado en `20`.
`z = x * y #`	**Sí**	Igual que la anterior, pero con ⟨ `expresion` ⟩ → ⟨ `termino` ⟩ ⟨ `op` ⟩ ⟨ `expresion` ⟩ y ⟨ `op` ⟩ → `*`, uno de los cinco operadores permitidos (reglas 17 a 21).
`salida temporal #`	**Sí**	Salida: ⟨ `instr` ⟩ → ⟨ `instr_salida` ⟩ → `salida` ⟨ `id` ⟩ `#`, con `temporal` como ⟨ `id` ⟩. El nombre de la variable cambia el lexema, no la estructura.

2.4. Pregunta 4 – Validación y detección de errores

Para cada caso aplicamos la gramática propuesta y revisamos si la cadena pertenece o no al lenguaje. En las que no pertenecen, indicamos qué regla falla y qué elemento falta o está mal ubicado.

Convención. Igual que en la sección 2.3, las cadenas sueltas se evalúan desde ⟨instr⟩. Los casos 1 a 3 también son válidos dentro de un programa completo, por ejemplo `Begin Programa p x = 10 # End`.

Cuadro 8: Análisis de las ocho cadenas de la pauta.

Caso	Cadena	¿Pertenece al lenguaje?	Justificación / error detectado
1	`x = 10 #`	**Sí**	Asignación simple: ⟨ `instr` ⟩ → ⟨ `asignacion` ⟩, con ⟨ `termino` ⟩ → ⟨ `num` ⟩ instanciado en `10`.
2	`z=x+y#`	**Sí**	Asignación con operación: ⟨ `expresion` ⟩ → ⟨ `termino` ⟩ ⟨ `op` ⟩ ⟨ `expresion` ⟩, con ⟨ `op` ⟩ → `+`. Los espacios no afectan porque no pertenecen al alfabeto.
3	`salida z #`	**Sí**	Salida: ⟨ `instr` ⟩ → ⟨ `instr_salida` ⟩ → `salida` ⟨ `id` ⟩ `#`, con ⟨ `id` ⟩ instanciado en `z`.
4	`x 10 #`	**No**	**Falta el operador `=`.** Después de `x`, la regla ⟨ `asignacion` ⟩ exige `=`, pero aparece el número `10`.
5	`z=+xy#`	**No**	**Operador mal ubicado: falta el operando izquierdo.** La expresión debe comenzar con un ⟨ `termino` ⟩ (⟨ `id` ⟩ o ⟨ `num` ⟩), y `+` no puede iniciarla. Parece notación prefija (`+ x y`), que esta gramática no admite.

Caso	Cadena	¿Pertenece al lenguaje?	Justificación / error detectado
6	`salida #`	**No**	**Falta el identificador.** La regla ⟨`instr_salida`⟩ exige un ⟨`id`⟩ entre `salida` y `#`, pero aparece directamente `#`.
7	`x = 20`	**No**	**Falta el terminador `#`.** La regla ⟨`asignacion`⟩ termina en `#`, pero la cadena se acaba después de `20`.

8	`Begin Programa`
	`prueba x = 5 #`
	`salida x # End`
		Sí	Programa completo: ⟨`programa`⟩ → `Begin Programa` ⟨`id`⟩ ⟨`lista_instr`⟩ `End`, con ⟨`id`⟩ = `prueba` y dos instrucciones. Las diagonales `/` del enunciado solo separan líneas; no son parte de la cadena, porque `/` es el operador de división.

Resultado global: 4 cadenas válidas (casos 1, 2, 3 y 8) y 4 cadenas inválidas (casos 4, 5, 6 y 7).

a). Análisis general

Los errores detectados se agrupan en cuatro categorías, todas de naturaleza *sintáctica* (ninguno de los casos analizados falla en el análisis léxico, porque todos los caracteres utilizados pertenecen al alfabeto):

1. **Omisión de un elemento obligatorio en una posición obligatoria.** Es el error del caso 4 (= ausente) y del caso 7 (# ausente). La gramática exige esos terminales en posiciones exactas; el analizador léxico los reconoce sin problema, pero el analizador sintáctico detecta que la estructura de la regla ⟨`asignacion`⟩ no se completa.
2. **Presencia de un elemento en una posición incorrecta.** Es el error del caso 5: el operador `+` ocupa el lugar donde la gramática exige un operando. La regla que no se puede aplicar es ⟨`termino`⟩ → ⟨`id`⟩ | ⟨`num`⟩.
3. **Omisión de un operando o de un argumento.** Es el error del caso 6: falta el identificador que se desea imprimir. Nótese que la gramática no permite instrucciones vacías en ningún punto: la lista de instrucciones requiere al menos una (⟨`lista_instr`⟩ → ⟨`instr`⟩), y toda instrucción exige sus elementos obligatorios.
4. **Interpretación del caso 8.** En la tabla del enunciado, las diagonales `/` solo separan las líneas del programa y no forman parte de la cadena. Si se leyeran como caracteres, la cadena sería inválida, porque `/` es el operador de división y solo puede aparecer entre dos operandos dentro de ⟨expresion⟩.

En la práctica, *el analizador léxico y el sintáctico detectan, cada uno en su nivel, los errores*: el primero garantiza que cada cadena esté formada por tokens válidos, y el segundo que esos tokens se organicen según las estructuras que define la GLC. La gramática, al ser no ambigua, permite además identificar *un único* punto de falla por cadena, lo que facilita el diagnóstico.

3. Extensión de la gramática

Construcción seleccionada: Opción C — Declaración de variables.

3.1. Nueva sintaxis propuesta

Agregamos al lenguaje la instrucción de **declaración de variables con tipo explícito**, que nos permite reservar un nombre y asignarle un tipo de dato antes de su uso:

`Sintaxis`	**Descripción**
`entero` *id* `#`	Declara la variable *id* como de tipo entero.
`real` *id* `#`	Declara la variable *id* como de tipo real.

La construcción se eligió por su encaje natural en la gramática: es una instancia más de la no terminal ⟨ `instr` ⟩, por lo que se integra en la regla ⟨ `instr` ⟩ → ⟨ `asignacion` ⟩ | ⟨ `instr_salida` ⟩ | ⟨ `declaracion` ⟩ | ⟨ `comentario` ⟩ sin modificar la estructura de los demás tipos de instrucción. A diferencia de las opciones A y B, que requieren introducir nuevas categorías sintácticas (condicionales y bucles, con sus delimitadores y sus condiciones), la opción C no altera la gramática del lenguaje base más que con cuatro reglas nuevas, y es coherente con el espíritu del programa de referencia, que reserva nombres de variables.

3.2. Modificaciones realizadas a la gramática

Cuadro 9: Modificaciones introducidas por la extensión creativa.

Elemento	**Modificación**
Nuevos terminales	`entero` y `real` (palabras reservadas del tipo de dato), que se incorporan a Σ. Queda: {Begin, Programa, End, salida, entero, real, =, #, +, -, *, /, %, ID, NUM, COM}.
Nuevos no terminales	⟨ `declaracion` ⟩ y ⟨ `tipo` ⟩, que se incorporan a V. El primero describe la instrucción completa; el segundo agrupa los dos tipos posibles.
Nuevas producciones	⟨ `instr` ⟩ → ⟨ `declaracion` ⟩ (regla 6)
	⟨ `declaracion` ⟩ → ⟨ `tipo` ⟩ ⟨ `id` ⟩ `#` (regla 10)
	⟨ `tipo` ⟩ → `entero` (regla 11)
	⟨ `tipo` ⟩ → `real` (regla 12)
Modificación de P	Se agregan las cuatro reglas anteriores: P pasa de 20 a 24 reglas.
Modificación de S	Ninguna: el símbolo inicial sigue siendo ⟨ `programa` ⟩.

3.3. Ejemplo de cadena válida

```
real temperatura #
```

y, como programa completo,

```
Begin Programa demo real temperatura # salida temperatura # End
```

Ambos son aceptados por la gramática extendida (véase la derivación de la sección 3.5).

3.4. Ejemplo de cadena inválida

```
entero 5 #
```

La cadena es **inválida** porque en la declaración de una variable se exige un *nombre* (un identificador) y no un valor numérico: la regla ⟨ `declaracion` ⟩ → ⟨ `tipo` ⟩ ⟨ `id` ⟩ `#` exige un ⟨ `id` ⟩ después del tipo. En este caso se encuentra el número `5` donde la gramática exige un identificador. Un segundo ejemplo de cadena inválida sería `real x` (sin el terminador `#`).

3.5. Derivación de ejemplo

$$
\begin{aligned}
\langle\,\texttt{instr}\,\rangle &\Rightarrow \langle\,\texttt{declaracion}\,\rangle \\
&\Rightarrow \langle\,\texttt{tipo}\,\rangle\ \langle\,\texttt{id}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{real}\ \langle\,\texttt{id}\,\rangle\ \texttt{\#} \\
&\Rightarrow \texttt{real temperatura \#}
\end{aligned}
$$

Como la extensión es una alternativa de ⟨ `instr` ⟩, la derivación se obtiene exactamente igual que las de la Pregunta 3, sólo que por la rama ⟨ `instr` ⟩ → ⟨ `declaracion` ⟩. La derivación completa del programa `Begin Programa demo real temperatura # salida temperatura # End` es:

$$
\begin{aligned}
\langle\,\texttt{programa}\,\rangle &\Rightarrow \texttt{Begin Programa}\ \langle\,\texttt{id}\,\rangle\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{instr}\,\rangle\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{declaracion}\,\rangle\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo}\ \langle\,\texttt{tipo}\,\rangle\ \langle\,\texttt{id}\,\rangle\ \texttt{\#}\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo real}\ \langle\,\texttt{id}\,\rangle\ \texttt{\#}\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo real temperatura \#}\ \langle\,\texttt{lista_instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo real temperatura \#}\ \langle\,\texttt{instr}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo real temperatura \#}\ \langle\,\texttt{instr_salida}\,\rangle\ \texttt{End} \\
&\Rightarrow \texttt{Begin Programa demo real temperatura \# salida}\ \langle\,\texttt{id}\,\rangle\ \texttt{\# End} \\
&\Rightarrow \texttt{Begin Programa demo real temperatura \# salida temperatura \# End}
\end{aligned}
$$

4. Conclusiones

Este laboratorio nos mostró que una gramática libre de contexto es una forma precisa de describir cómo está construido un lenguaje, y que su valor está en poder comprobarlo con derivaciones, árboles y cadenas de prueba. A partir del programa de referencia llegamos a una gramática de 24 reglas que lo genera completo, produce las derivaciones por la izquierda pedidas y clasifica correctamente las ocho cadenas de la Pregunta 4 (cuatro válidas y cuatro inválidas), indicando en cada error qué elemento falta o está mal ubicado.

Aprendimos a distinguir entre el análisis léxico y el sintáctico: separarlos, con `ID` y `NUM` como clases léxicas, mantuvo la gramática pequeña. La recursión por la derecha y la ausencia de producciones vacías evitaron la ambigüedad y los programas sin instrucciones. Como limitación, la gramática no modela precedencia ni asociatividad de operadores, y la extensión de declaración de variables es solo sintáctica: no controla que una variable se declare antes de usarse.

Con JFLAP 7.1 aprendimos que la herramienta solo admite símbolos de un carácter, por lo que codificamos las palabras reservadas y documentamos la correspondencia para mantener la coherencia con el informe.

Anexo A: evidencia en JFLAP 7.1

La GLC fue cargada en JFLAP 7.1 mediante el archivo `GLC_Lab3.jff`. Este archivo y `GLC_Lab3_instr.jff` están en el repositorio de GitHub.

Repositorio de GitHub: `https://github.com/saezjose/Lab03_Teoria_Computacion`.

Para reproducir la evidencia:

1. Abrir JFLAP 7.1 y elegir **File** → **Open**, o bien arrastrar el archivo `GLC_Lab3.jff` sobre la ventana.
2. En el panel superior de la ventana aparece la **tabla de la gramática** con las 24 reglas tal como quedaron cargadas, en la codificación de un carácter descrita en la Tabla 5.
3. Para construir el árbol sintáctico se usa el menú **Input** → **Brute Force Parse**: en el campo *Input* de la parte inferior se escribe la cadena de tokens y se pulsa **Start**. JFLAP indica el resultado en una etiqueta (*"String accepted!"* o *"String rejected!"*) y el árbol se dibuja en el lienzo al pulsar **Step**.
4. Con el archivo `GLC_Lab3.jff` (símbolo inicial `S`) se analiza un programa completo mínimo: `bpww=w#e` (equivalente a `Begin Programa demo w = w # End`). Para analizar una sola instrucción se abre el archivo auxiliar `GLC_Lab3_instr.jff`, que tiene las mismas 24 reglas con la primera producción `I → A` (por eso el símbolo inicial de la herramienta es `I`), y se escribe `w=w+w#` (equivalente a `z = x + y #`).

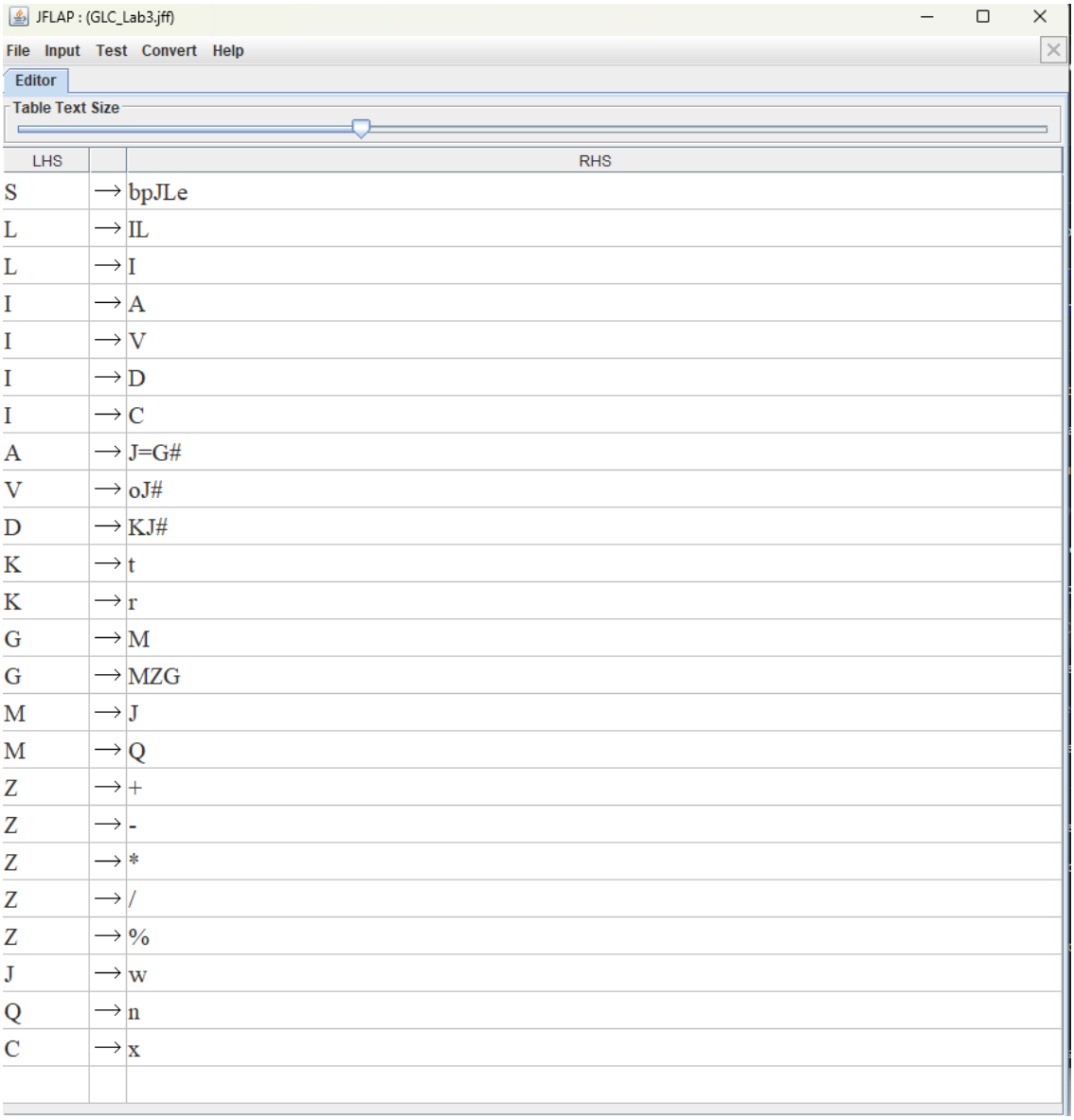


Figura 4: Editor de gramáticas de JFLAP 7.1 con las 24 reglas cargadas desde `GLC_Lab3.jff`.

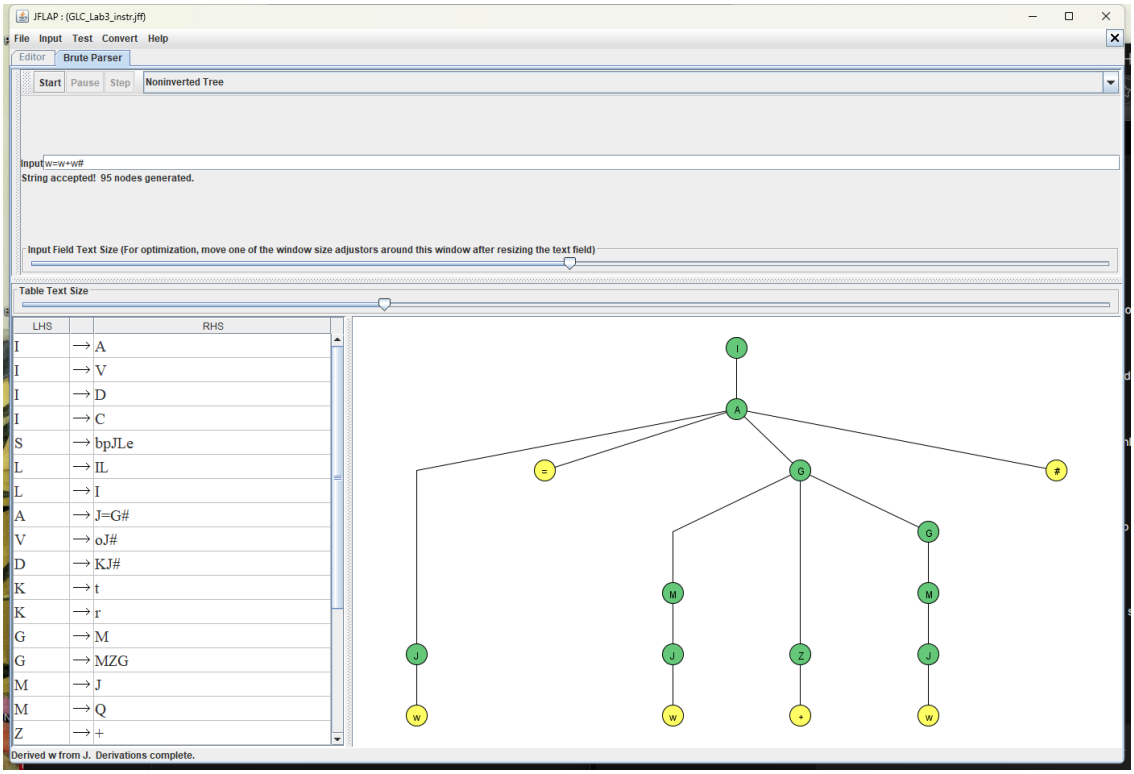


Figura 5: Árbol sintáctico construido por JFLAP 7.1 (**Input** → **Brute Force Parse**) para la cadena de tokens `w=w+w#`, equivalente a la cadena `z = x + y #` de la Figura 2. La etiqueta de JFLAP indica *"String accepted!"*.

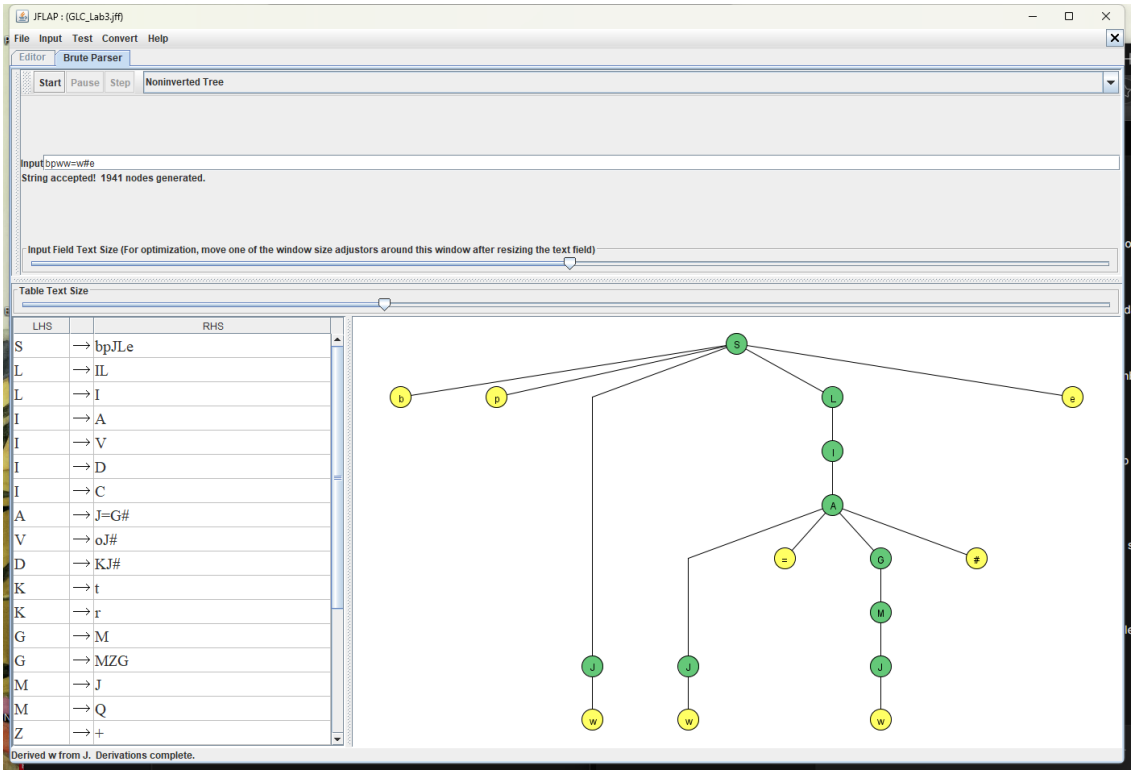


Figura 6: Árbol sintáctico del programa completo construido por JFLAP 7.1 (`bpww=w#e`, con el símbolo inicial `S`). El programa es el mínimo posible; con el programa de la Figura 3 la búsqueda por fuerza bruta de JFLAP 7.1 no concluye en un tiempo razonable.

Anexo B: uso de inteligencia artificial

Uso de IA

Se utilizó IA (asistente de programación) como apoyo en este laboratorio: con su ayuda se generaron las estructuras para el texto del informe y los árboles sintácticos.

Los integrantes del grupo revisaron los resultados: probaron la gramática en JFLAP 7.1, comprobaron las derivaciones y las cadenas de la Pregunta 4, y aprobaron el contenido final.

Referencias

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.ª ed.). Addison-Wesley. Capítulos 3 (análisis léxico) y 4 (análisis sintáctico).

[2] Rodger, S. H. (s. f.). *JFLAP* (versión 7.1) [Software]. Duke University. `https://www.jflap.org`

[3] Sipser, M. (2012). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning. Capítulo 2 (lenguajes libres de contexto).

[4] Tutorialspoint. (s. f.). *Automata theory – Applications of context-free grammar*. Recuperado el 24 de octubre de 2026, de `https://www.tutorialspoint.com/automata_theory/applications_of_context_free_grammar.htm`